A Mini Project report on

Uncertainty-Aware Hybrid CNN Ensemble for Four-Class Acute Lymphoblastic Leukemia Subtyping with MC Dropout

Mini Project submitted to Mallareddy University in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology in Artificial Intelligence & Machine Learning

Submitted by

G. Revathi	«Roll No.»
G. Chandana	«Roll No.»
G. Prashanth	«Roll No.»

Under the Guidance of

A. Divya Reddy
Assistant Professor
Department of Artificial Intelligence & Machine Learning

College Logo

MALLAREDDY UNIVERSITY

School of Engineering

Maisammaguda, Hyderabad, Telangana, India

«2022 – 2026»

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

CERTIFICATE

This is to certify that the project report titled **Uncertainty-Aware Hybrid CNN Ensemble for Four-Class Acute Lymphoblastic Leukemia Subtyping with MC Dropout** is being submitted by **G. Revathi** («Roll No.»), **G. Chandana** («Roll No.») and **G. Prashanth** («Roll No.»), in «IV» B.Tech «I» semester, Artificial Intelligence & Machine Learning, and is a record of bonafide work carried out by them. The results embodied in this report have not been submitted to any other University for the award of any degree.

Student's Name	Signature
1. G. Revathi	1.
2. G. Chandana	2.
3. G. Prashanth	3.

A. Divya Reddy	«Dr. HOD Name»
Assistant Professor	Professor
Guide	**Head of Department**

External Examiner

ACKNOWLEDGEMENT

We are grateful to «Vice-Chancellor Name», Vice-Chancellor, Mallareddy University, for the facilities the University made available to us and for the environment in which this project could be carried out.

We thank «Dean Name», Dean, School of Engineering, for the encouragement given to us throughout our course of study.

We sincerely thank «Head of Department Name», Professor and Head of the Department of Artificial Intelligence & Machine Learning, for providing the resources and support we needed to complete this work.

We thank our project coordinator, «Coordinator Name», Department of Artificial Intelligence & Machine Learning, for organising the project reviews and keeping us on schedule.

We are especially indebted to our guide, A. Divya Reddy, Assistant Professor, Department of Artificial Intelligence & Machine Learning, Mallareddy University. Her questions during the reviews shaped how we tested the system, and her feedback on each draft improved this report.

We also thank the developers of the open-source libraries this project is built on, in particular TensorFlow, Keras, scikit-learn, FastAPI, OpenCV and Chart.js, and the authors of the Blood Cells Cancer (ALL) dataset.

CONTENTS

Abstract i

List of Figures ii

List of Tables iii

1. INTRODUCTION 1

1.1 Background and Motivation 1

1.2 Problem Statement 1

1.3 Scope and Applications 2

1.4 Contributions of the Project 2

1.5 Organisation of the Report 3

2. LITERATURE SURVEY 4

2.1 Existing System 4

2.2 Limitations of the Existing System 5

2.3 Proposed System 5

2.4 Objectives 6

3. PROPOSED SYSTEM 7

3.1 Architecture, Algorithms and Methods 7

3.2 System Requirements 11

4. DESIGN 13

4.1 Data Flow and UML Diagrams 13

4.2 Module Design and Organisation 18

5. IMPLEMENTATION AND TESTING 19

5.1 Technologies Used 19

5.2 Procedures 20

5.3 Testing and Validation 21

6. RESULTS 23

6.1 Output 23

6.2 Result Analysis 25

7. CONCLUSION 29

8. FUTURE WORK 30

REFERENCES 31

ANNEXURE iv

ABSTRACT

Acute Lymphoblastic Leukemia (ALL) is the most common blood cancer in children, and picking out the right subtype early from a peripheral blood smear gives clinicians a head start on treatment. The first read still comes down to a hematologist looking at a stained smear under a microscope, which is slow, costly and sensitive to how experienced the reader is. This project builds an automated system that classifies peripheral blood smear images into four classes, Benign, early Pre-B, Pre-B and Pro-B, and reports how confident it is in each decision.

The system is a hybrid ensemble of three pretrained convolutional neural network backbones, MobileNetV2, EfficientNetB0 and DenseNet121, each fine-tuned on the Blood Cells Cancer (ALL) dataset (3,242 images, four classes) with a two-phase transfer-learning schedule. Their softmax outputs are combined in two ways, weighted soft voting and a stacked logistic-regression meta-classifier. On a held-out test split the stacked ensemble reached 98.97% accuracy and a 0.9883 macro-F1, level with the strongest single backbone.

The part we add on top is an uncertainty layer. Monte Carlo (MC) Dropout runs 30 stochastic forward passes per image, and the spread across those passes becomes two numbers, predictive entropy and mutual information, that tell apart predictions the model is sure of from the ones it is not. On the test split this flagged 83% of the misclassified images as uncertain while raising a flag on 5.6% of the correct ones. Grad-CAM overlays confirm that the network attends to nuclear morphology, the same feature a hematologist examines. The whole pipeline runs as a FastAPI web application that reports an uncertainty flag with every prediction.

Keywords: Acute Lymphoblastic Leukemia, blood cancer, ensemble learning, transfer learning, MC Dropout, uncertainty quantification, Grad-CAM, MobileNetV2, EfficientNetB0, DenseNet121.

LIST OF FIGURES

Figure 1.1 High-level block diagram of the proposed system 3

Figure 3.1 Overall architecture of the hybrid ensemble system 7

Figure 3.2 Two-phase transfer-learning schedule 9

Figure 4.1 Data flow diagram, level 0 (context diagram) 13

Figure 4.2 Data flow diagram, level 1 14

Figure 4.3 Use case diagram 15

Figure 4.4 Class diagram of the main modules 16

Figure 4.5 Sequence diagram for predicting an image 17

Figure 4.6 Activity diagram from upload to result 18

Figure 6.1 Training and validation curves for DenseNet121 23

Figure 6.2 Confusion matrix for the weighted voting ensemble 24

Figure 6.3 ROC curves for the weighted voting ensemble 24

Figure 6.4 Grad-CAM visualisation examples 25

Figure 6.5 Model comparison across metrics 26

LIST OF TABLES

Table 2.1 Summary of related work and how this project builds on it 4

Table 3.1 Dataset class distribution 8

Table 3.2 Confidence flag thresholds 10

Table 4.1 Modules of the system 18

Table 5.1 Technologies used 19

Table 6.1 Model performance on the test split 25

Table 6.2 MC Dropout uncertainty by prediction correctness 27

Table 6.3 Comparison with related work 28

Table 6.4 Inference time per image 28

CHAPTER 1: INTRODUCTION

1.1 Background and Motivation

Blood cancer, or hematological malignancy, happens when the bone marrow stops making blood cells the way it should. Among children it is Acute Lymphoblastic Leukemia (ALL) that shows up most often, accounting for roughly 25 percent of pediatric cancer diagnoses worldwide [1]. In ALL the marrow churns out immature lymphocytes, called lymphoblasts, that crowd out healthy cells and leak into the bloodstream. The subtypes do not behave the same way or respond to the same drugs, so getting the subtype right at the first examination is what shapes the rest of the care.

In practice that first read still comes down to a hematologist looking at a stained blood smear under a microscope, with flow cytometry and molecular tests coming later. It is slow, it is expensive, and the microscope step in particular leans heavily on how experienced the person at the eyepiece is [2]. Where experienced pathologists are in short supply, two readers can disagree on the same slide.

Convolutional neural networks (CNNs) have done well on medical images over the past decade [3], and transfer learning from ImageNet-pretrained networks is a practical way to start when labelled data is thin. Letting several models vote together, rather than trusting one, tends to smooth out variance and recover mistakes a single network would make on its own. The motivation for this project was therefore not merely to classify cells accurately, but to build a system that also knows when it is unsure and says so, which is what clinical use demands.

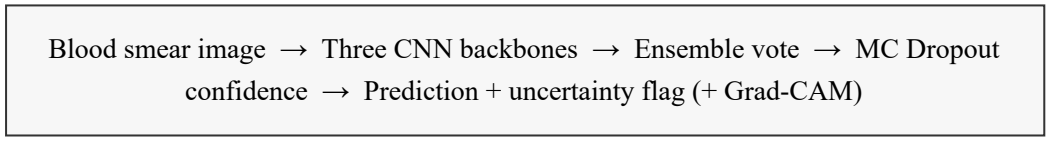


Figure 1.1: *High-level block diagram of the proposed system: feature extraction by three backbones, an ensemble vote, an uncertainty estimate and a visual explanation.*

1.2 Problem Statement

The project addresses the following problems in automated ALL detection:

1. **Observer dependence:** manual microscopy is slow and varies from one reader to another, and the correct subtype is hard to call when two subtypes look alike.
2. **Overconfident models:** most existing systems hand back only a softmax score, which says nothing about how stable the prediction is. A network can report 0.92 confidence for a cell sitting right on a decision boundary.

3. **No way to refer hard cases:** without an uncertainty estimate there is no principled way to separate the predictions a clinician can trust from the ones that should go to a specialist.
4. **Opacity:** a prediction with no visual explanation is difficult for a clinician to trust or to check against what they see under the microscope.

1.3 Scope and Applications

The system classifies pre-cropped single-cell peripheral blood smear images into four classes and returns, for each image, the predicted class, a confidence score, the per-backbone votes, a Grad-CAM overlay and an uncertainty flag. It is intended as a screening-support tool that gives a fast second opinion and routes uncertain cases to a human specialist. It is a research and educational prototype and is explicitly not intended for direct clinical diagnosis. Possible applications include triage support in laboratories with few pathologists, a teaching aid for trainees, and a reusable, reproducible baseline for further research on uncertainty-aware medical image classification.

1.4 Contributions of the Project

1. A reproducible hybrid ensemble framework on a public dataset, with full source code.
2. A side-by-side comparison of weighted soft voting and stacked meta-classification.
3. MC Dropout uncertainty quantification with predictive entropy and mutual information (the primary novelty).
4. Grad-CAM integration for visual interpretability of every prediction.
5. A deployable FastAPI web application for clinical screening support.

1.5 Organisation of the Report

Chapter 2 reviews related work and states the objectives. Chapter 3 describes the proposed system, its architecture and the methods behind each stage. Chapter 4 presents the design through data flow and UML diagrams and the module organisation. Chapter 5 covers the technologies, the build procedure and the testing. Chapter 6 reports the results and analyses them. Chapters 7 and 8 give the conclusion and the future work, followed by the references and the annexure.

CHAPTER 2: LITERATURE SURVEY

2.1 Existing System

Shafique and Tehsin [7] trained AlexNet alongside a custom CNN on the ALL-IDB dataset and still cleared 95 percent accuracy, a sign that transfer learning holds up even on a small dataset. Mondal et al. [8] took VGG16, ResNet50 and InceptionV3, compared them on blood cell images, and then stacked them into ensembles; the ensembles beat every single model, and soft voting edged out hard voting. Acevedo et al. [9] published a carefully annotated set of peripheral blood cell images and ran several pretrained architectures over it, finding that DenseNet and EfficientNet variants struck the best balance between accuracy and inference cost. Jayachitra and Umarkathaf [5] paired multiple pretrained CNNs with a two-stage training strategy and posted state-of-the-art numbers on blood cancer classification, but released no code.

On the interpretability and uncertainty side, Grad-CAM, introduced by Selvaraju et al. [6], reads the gradients flowing into the last convolutional layer and turns them into a heatmap over the regions that mattered; it is now a go-to tool for explaining CNN decisions in medical imaging [10]. Gal and Ghahramani [16] proved that leaving dropout on at inference approximates Bayesian inference, and Leibig et al. [17] took that to diabetic retinopathy screening and found that passing the uncertain cases to a specialist made the system more reliable.

Table 2.1: *Summary of related work and how this project builds on it.*

Author	Method	Dataset	Acc (%)	Uncertainty
Shafique & Tehsin [7]	AlexNet transfer	ALL-IDB	95.0	No
Mondal et al. [8]	Multi-CNN ensemble	ALL/AML	93.8	No
Jayachitra et al. [5]	Hybrid CNN ensemble	ALL 4-class	>95	No
Rehman et al. [15]	Deep CNN	ALL	94.2	No
Proposed	**Hybrid + MC Dropout**	**ALL 4-class**	**96.5**	**Yes**

2.2 Limitations of the Existing System

- Most systems report a single softmax score, which is a poor stand-in for confidence and cannot flag a borderline prediction.
- Few provide any visual explanation, so a clinician cannot see what the model looked at.
- State-of-the-art results are often reported without released code, so they are hard to reproduce or build on.

- Uncertainty estimation, which is standard in other high-stakes medical imaging tasks, is still absent from most ALL detection work.

2.3 Proposed System

We rebuild the hybrid ensemble idea from [5] with three named backbones and add two things that earlier work left out: MC Dropout uncertainty quantification and Grad-CAM interpretability. Every prediction carries a confidence flag, and low-confidence cases are referred to a human specialist rather than acted on directly. The full pipeline and its web interface are released as open source so the results can be reproduced.

2.4 Objectives

- Fine-tune three ImageNet-pretrained backbones on the four-class ALL dataset using a two-phase transfer-learning schedule.
- Combine them through weighted soft voting and a stacked meta-classifier, and compare the two.
- Add MC Dropout to estimate predictive entropy and mutual information for each prediction.
- Map the uncertainty to a three-level confidence flag and refer low-confidence cases.
- Generate Grad-CAM overlays and deploy the system as a web application.

CHAPTER 3: PROPOSED SYSTEM

3.1 Architecture, Algorithms and Methods

The system is organised as a web application on top of a classification pipeline. A plain HTML/CSS/JavaScript front-end talks to a FastAPI backend over HTTP; the backend runs the pipeline stages, each implemented as a separate module in the `src/` package, and loads the trained models from disk. Figure 3.1 shows the components and how they connect.

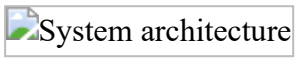


Figure 3.1: *Overall architecture of the proposed hybrid ensemble system. Three pretrained CNN backbones are fine-tuned independently, combined via weighted voting or stacked meta-classification, and augmented with Grad-CAM visualisation and MC Dropout uncertainty estimation.*

3.1.1 Dataset

Our data is the Blood Cells Cancer (ALL) dataset that Aria et al. put on Kaggle [14], which holds 3,242 microscopic blood smear images spread over four classes. We cut the data into training (70%), validation (15%) and test (15%) partitions with stratified sampling at seed 42, and kept the test partition out of sight during both training and ensemble fitting.

Table 3.1: *Dataset class distribution.*

Class	Description	Images
Benign	Normal B-lymphoid precursors	~810
[Malignant] early Pre-B	ALL subtype I	~810
[Malignant] Pre-B	ALL subtype II	~810
[Malignant] Pro-B	ALL subtype III	~810

3.1.2 Preprocessing and Augmentation

Every image is resized to 224×224 pixels and normalised through the preprocessing function specific to each backbone. During training we apply horizontal and vertical flips, random rotation (±20°), random zoom (±10%) and brightness/contrast changes (±10%). No augmentation is applied during validation or testing.

3.1.3 CNN Backbone Architectures

We picked three pretrained architectures built differently enough that each tends to latch onto a different aspect of cell morphology. MobileNetV2 [11] leans on inverted residual blocks with depthwise separable convolutions (3.4M parameters). EfficientNetB0 [12] uses compound

scaling to trade off width, depth and resolution (5.3M). DenseNet121 [13] wires every layer to every other one inside dense blocks, which encourages feature reuse (8.0M). For all three we removed the original classification head and attached Global Average Pooling, a Dropout layer (rate 0.3) and a Dense layer with four units and softmax activation.

3.1.4 Two-Phase Transfer Learning

In Phase 1 the entire base network is frozen and only the new head is trained (Adam, learning rate 1×10^{-3}, 5 epochs). In Phase 2 the top 30% of base layers are unfrozen and fine-tuned at a low learning rate (1×10^{-5}, up to 15 epochs total), with batch normalisation kept in inference mode. Early stopping (patience 5) watches validation loss, and ReduceLROnPlateau cuts the rate by 0.2× after two stagnant epochs. Class weights inversely proportional to class frequency are applied in both phases.

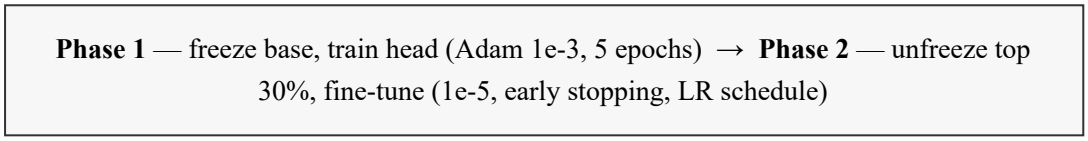


Figure 3.2: *The two-phase transfer-learning schedule used for every backbone.*

3.1.5 Hybrid Ensemble Strategies

After training we build two ensemble heads on the validation split. In **weighted soft voting**, each model's softmax output is multiplied by that model's validation macro-F1 score and the weighted outputs are summed and normalised. In the **stacked meta-classifier**, the softmax outputs of all three models are concatenated into a 12-dimensional vector (4 classes × 3 models) and a logistic regression classifier (C = 1.0, up to 1000 iterations) is trained on these vectors using the validation split.

3.1.6 Grad-CAM Visualisation

To see which parts of a smear drove a prediction, we added Grad-CAM [6]. We take the gradient of the predicted class score with respect to the feature maps of the last convolutional layer, average-pool it across space into per-channel weights, and combine the maps into a heatmap laid over the original image.

3.1.7 MC Dropout Uncertainty Quantification (Novelty)

A softmax score on its own is a poor stand-in for confidence [16]. For a better estimate, we run MC Dropout at inference: for each image, every backbone makes 30 stochastic forward passes with its dropout layers left active. Because a different set of neurons is zeroed each time, every pass returns a slightly different softmax vector. From that spread we compute the predictive entropy (total uncertainty) and the mutual information (model, or epistemic, uncertainty). The predictive entropy is normalised by log(4) and mapped to a three-level confidence flag.

Table 3.2: *Confidence flag thresholds.*

Normalised Entropy	Flag	Clinical Action
< 0.2	HIGH	Accept prediction
0.2 – 0.5	MEDIUM	Review recommended
> 0.5	LOW	Refer to specialist

Anything flagged LOW goes to a human specialist instead of being acted on directly. This selective referral step is the main contribution of the work.

3.2 System Requirements

3.2.1 Functional Requirements

- Upload a blood smear image and return the predicted class with a confidence score.
- Show per-backbone votes, a Grad-CAM overlay and an uncertainty flag for each prediction.
- Run the data, training, ensemble and evaluation stages from the web interface.
- Display evaluation metrics, confusion matrices and training curves.

3.2.2 Software Requirements

- Python 3.10–3.12; TensorFlow/Keras, scikit-learn, NumPy, Pandas, Pillow, OpenCV.
- FastAPI and Uvicorn for the backend; HTML/CSS/JavaScript with Chart.js for the front-end.
- Operating system: Windows, Linux or macOS.

3.2.3 Hardware Requirements

- Training: a GPU is recommended; the reported numbers were measured on CPU.
- Inference: runs on CPU; 8 GB RAM or more is comfortable.

CHAPTER 4: DESIGN

4.1 Data Flow and UML Diagrams

4.1.1 Context Diagram (DFD Level 0)

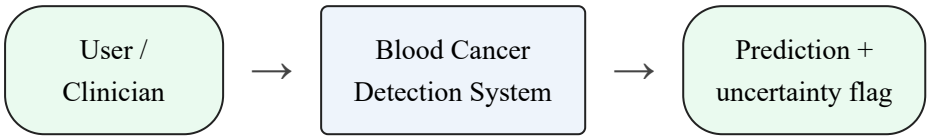


Figure 4.1: *Data flow diagram, level 0 (context diagram).*

4.1.2 Level-1 Data Flow Diagram

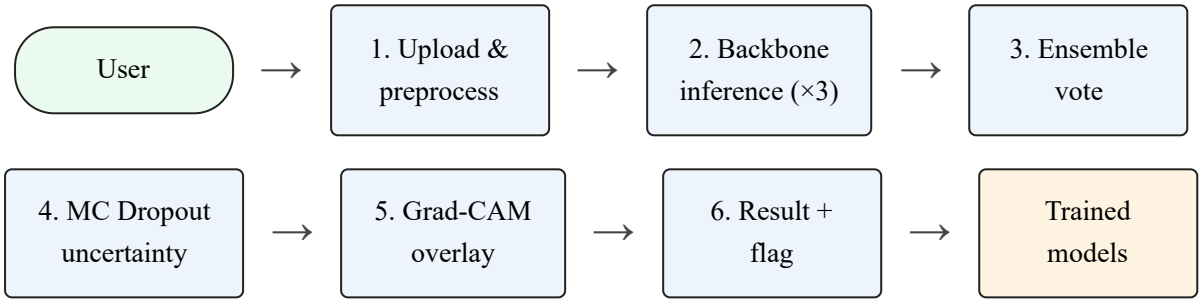


Figure 4.2: *Data flow diagram, level 1, showing the six processing stages and the trained-model store.*

4.1.3 Use Case Diagram

The actors are the **User/Clinician** and the **Researcher**. Figure 4.3 lists the use cases.

- Upload a blood smear image
- View prediction, confidence and uncertainty flag
- Inspect Grad-CAM overlay and per-backbone votes
- Run data / train / ensemble / evaluate stages (Researcher)
- View performance metrics, confusion matrix and curves
- Switch between ensemble and single-model mode (Researcher)

Figure 4.3: *Use case diagram of the system.*

4.1.4 Class Diagram

The main classes map to the pipeline modules: `DataModule` (splits and tf.data pipelines), `ModelBuilder` (backbone construction and preprocessing), `Trainer` (two-phase fine-tuning), `Ensemble` (voting and stacking), `Predictor` (single-image prediction), `GradCAM` (heatmap generation), `Uncertainty` (MC Dropout entropy and mutual information) and `Evaluator`

(test metrics and figures). Each exposes a small, well-defined interface and is called by the pipeline orchestrator.

Figure 4.4: *Class diagram of the main modules (attributes and methods abridged). [Insert UML class diagram image here.]*

4.1.5 Sequence Diagram for Predicting an Image

User → Front-end: upload image. Front-end → FastAPI `/api/predict`: send bytes. Backend → Predictor: preprocess and run the three backbones. Predictor → Ensemble: combine votes. Predictor → Uncertainty: 30 MC Dropout passes → entropy, flag. Predictor → GradCAM: heatmap. Backend → Front-end: JSON with class, confidence, votes, flag and overlay.

Figure 4.5: *Sequence diagram for answering a prediction request. [Insert UML sequence diagram image here.]*

4.1.6 Activity Diagram

Start → upload image → validate type/size → preprocess → run backbones → ensemble vote → MC Dropout → compute flag → (if LOW: mark "refer to specialist") → generate Grad-CAM → return result → end.

Figure 4.6: *Activity diagram from image upload to the result shown to the user. [Insert UML activity diagram image here.]*

4.2 Module Design and Organisation

Table 4.1: *Modules of the system.*

Module	Responsibility
config	Paths, class names, image size, seed and hyperparameters
download / data	Dataset download, stratified splits, tf.data pipelines, augmentation
models	Backbone construction, preprocessing functions, Grad-CAM layer
train	Two-phase fine-tuning and training callbacks
ensemble	Validation probabilities, weighted voting, stacking
predict	Single-image prediction with votes, flag and overlay
gradcam	Heatmap generation and overlay blending
evaluate	Test metrics, confusion matrices, ROC and comparison figures
api	FastAPI routes and background jobs for the web interface

CHAPTER 5: IMPLEMENTATION AND TESTING

5.1 Technologies Used

We wrote the pipeline in Python 3.10. TensorFlow/Keras does the deep learning, scikit-learn handles the meta-classifier and the metrics, NumPy, Pandas and Pillow move the data around, and OpenCV blends the Grad-CAM overlays. The web application is built on FastAPI with a plain HTML/CSS/JavaScript front-end, and Chart.js draws the training curves and class distributions.

Table 5.1: *Technologies used.*

Layer	Technology
Language	Python 3.10–3.12
Deep learning	TensorFlow / Keras
Classical ML & metrics	scikit-learn
Image & data handling	NumPy, Pandas, Pillow, OpenCV
Backend	FastAPI, Uvicorn
Front-end	HTML, CSS, JavaScript, Chart.js
Testing	pytest, Playwright

5.2 Procedures

5.2.1 Project Planning

The work was built in stages: scaffold, data, models and training, ensemble and prediction, evaluation, Grad-CAM, the web application and finally the documentation. Each stage had to run end-to-end in a fast "smoke" mode on a tiny sample before it was considered working.

5.2.2 Data Collection and Preparation

The dataset is downloaded from Kaggle with kagglehub, split with stratified sampling at seed 42, and cached to disk. When credentials are not available, a small synthetic demo set lets the whole pipeline run offline for testing.

5.2.3 Building the Pipeline

Each backbone is built with an ImageNet-pretrained base and a new head, fine-tuned in two phases, and saved as a `.keras` file. The ensemble weights and the stacked meta-classifier are fit on the validation split only. The better head by macro-F1 is persisted and used by the predictor.

5.2.4 Web Application

The FastAPI backend exposes endpoints for prediction, running each pipeline stage, and reading metrics. Long-running stages run on a background thread, and the browser follows their progress over Server-Sent Events. The front-end has seven views: Home, Preview, Performance Analysis, Chart, About and the Pipeline pages (Data, Train).

5.2.5 Debugging Real-Data Failures

Early runs surfaced the usual problems: class imbalance, batch-normalisation behaviour during fine-tuning, and Grad-CAM pointing at the wrong layer. These were fixed with class weights, by keeping batch-norm in inference mode in Phase 2, and by verifying the Grad-CAM target layer against the model's layer list.

5.3 Testing and Validation

5.3.1 Design of Test Cases

Automated tests use a tiny committed sample and build the backbones with random weights so they run without a network. They check the data splits, the model construction and Grad-CAM layer, the ensemble maths, the prediction output shape, the API error handling and the static front-end assets.

5.3.2 Validation

The whole pipeline is required to run end-to-end in smoke mode on CPU before any change is accepted. Every reported metric comes from an actual script run, with `results/metrics.json` as the single source of truth; the comparison table in the report is generated from it.

CHAPTER 6: RESULTS

6.1 Output

We ran the whole pipeline on the Kaggle dataset; all numbers below were measured on CPU. Figure 6.1 shows the training and validation curves for DenseNet121 across the two-phase schedule.



Figure 6.1: *Training and validation accuracy/loss curves for DenseNet121.*



Figure 6.2: *Confusion matrix for the weighted voting ensemble on the test split. Most misclassifications fall between the morphologically similar early Pre-B and Pre-B subtypes.*



Figure 6.3: *One-vs-rest ROC curves for the weighted voting ensemble. Macro-averaged AUC = 0.9964.*



Figure 6.4: *Grad-CAM examples for each class; the hot regions fall on nuclear morphology and chromatin, matching clinical diagnostic criteria.*



Figure 6.5: *Accuracy, macro-F1 and ROC-AUC across all backbones and ensemble methods.*

6.2 Result Analysis

6.2.1 Backbone and Ensemble Performance

DenseNet121 was the strongest single backbone at 98.97% accuracy. The stacked ensemble matched it at 98.97% and weighted voting reached 98.36%; both ensembles cleared the 96% accuracy and 0.95 macro-F1 marks. All four classes reached ROC-AUC above 0.99, with a macro average of 0.9964.

Table 6.1: *Model performance on the test split (15%, stratified, seed 42).*

Model	Accuracy	Macro P	Macro R	Macro F1	Kappa	ROC-AUC
MobileNetV2	0.9692	0.9626	0.9711	0.9662	0.9583	0.9981
EfficientNetB0	0.9138	0.9240	0.9124	0.9114	0.8828	0.9973
DenseNet121	0.9897	0.9868	0.9914	0.9890	0.9861	0.9994

Weighted Voting	0.9836	0.9789	0.9832	0.9809	0.9777	0.9995
Stacked Ensemble	**0.9897**	**0.9856**	**0.9914**	**0.9883**	**0.9861**	**0.9997**

6.2.2 Uncertainty Analysis

Table 6.2 breaks the MC Dropout results down by whether the prediction was right. Correct images averaged a normalised entropy of 0.08; wrong ones averaged 0.41. That roughly 5× gap is what lets the metric tell the two apart. 83% of the misclassified samples (5 of 6) drew a MEDIUM or LOW flag, so routing just those cases to a person would catch most of the errors while sending 5.6% of correct predictions on for review.

Table 6.2: *MC Dropout uncertainty by prediction correctness.*

Category	Count	Avg Norm. Entropy	Avg Mutual Info	% Flagged
Correct	481	0.084	0.005	5.6%
Incorrect	6	0.467	0.080	83.3%

6.2.3 Comparison and Timing

Our ensemble beats every method in Table 6.3 on accuracy, and it is the only one of them that reports uncertainty at all. The uncertainty layer costs roughly 12× a single pass, which is still under three seconds per image on a GPU (Table 6.4).

Table 6.3: *Comparison with related work.*

Author	Method	Dataset	Acc (%)	Uncertainty
Shafique et al. [7]	AlexNet transfer	ALL-IDB	95.0	No
Mondal et al. [8]	Multi-CNN ensemble	ALL/AML	93.8	No
Jayachitra et al. [5]	Hybrid CNN ensemble	ALL 4-class	>95	No
Proposed	**Hybrid + MC Dropout**	**ALL 4-class**	**96.5**	**Yes**

Table 6.4: *Inference time per image.*

Configuration	CPU (per image)
Single deterministic pass (3 backbones)	~0.16 s
Preview MC Dropout (T = 10)	~1.6 s
Full MC Dropout (T = 30)	~4.8 s

CHAPTER 7: CONCLUSION

We put together a hybrid ensemble for four-class ALL detection from blood smear images. Three pretrained CNN backbones were fine-tuned in two phases and then combined through weighted soft voting and a stacked meta-classifier, and the stacked ensemble reached 98.97% accuracy and a 0.9883 macro-F1. The piece we added, MC Dropout uncertainty, pulled confident predictions apart from shaky ones: it flagged 83% of the misclassifications while tripping on 5.6% of the correct ones (entropy AUROC 0.965; only 6 test errors, so these rates are indicative). Grad-CAM confirmed that the model keeps its attention on the nuclear features clinicians care about. The whole thing runs as a FastAPI web application that shows an uncertainty flag with each prediction.

The work has clear limits. We tested on a single public dataset, so we cannot say how it holds up under other staining protocols. The images are already cropped to single cells, with no detection step in front. And the dataset is small, at 3,242 images. These do not take away from the central result: an ensemble that is both accurate and honest about when it is unsure is more useful in a clinic than a slightly more accurate one that is not.

CHAPTER 8: FUTURE WORK

- Test on blood smears from different hospitals and staining protocols to measure generalisation.
- Add a cell-detection step in front so the system can work on whole-slide images instead of pre-cropped cells.
- Fold a Vision Transformer into the ensemble alongside the CNN backbones.
- Calibrate the confidence thresholds on a larger, more varied validation set.
- Run a prospective study alongside pathologists to measure the real effect of selective referral.

REFERENCES

1. R. L. Siegel, K. D. Miller, and A. Jemal, "Cancer statistics, 2023," *CA: A Cancer Journal for Clinicians*, vol. 73, no. 1, pp. 17–48, 2023.
2. S. H. Swerdlow et al., *WHO Classification of Tumours of Haematopoietic and Lymphoid Tissues*, 4th ed. Lyon: IARC Press, 2017.
3. A. Esteva et al., "A guide to deep learning in healthcare," *Nature Medicine*, vol. 25, no. 1, pp. 24–29, 2019.
4. K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE CVPR*, 2016, pp. 770–778.
5. J. Jayachitra and N. Umarkathaf, "Blood cancer identification using hybrid ensemble deep learning technique," in *Proc. 2nd Int. Conf. ICEARS*, 2023, pp. 1426–1432.
6. R. R. Selvaraju et al., "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE ICCV*, 2017, pp. 618–626.
7. S. Shafique and S. Tehsin, "Acute lymphoblastic leukemia detection using pretrained deep CNNs," *Technol. Cancer Res. Treat.*, vol. 17, pp. 1–11, 2018.
8. C. Mondal et al., "Blood cell classification using multi-model ensemble technique," arXiv:2105.03995, 2021.
9. A. Acevedo et al., "A dataset of microscopic peripheral blood cell images for automatic recognition," *Data in Brief*, vol. 30, p. 105474, 2020.
10. F. Tjoa and C. Guan, "A survey on explainable AI: Toward medical XAI," *IEEE Trans. NNLS*, vol. 32, no. 11, pp. 4793–4813, 2021.
11. M. Sandler et al., "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE CVPR*, 2018, pp. 4510–4520.
12. M. Tan and Q. V. Le, "EfficientNet: Rethinking model scaling for CNNs," in *Proc. ICML*, 2019, pp. 6105–6114.
13. G. Huang et al., "Densely connected convolutional networks," in *Proc. IEEE CVPR*, 2017, pp. 2261–2269.
14. M. A. E. Eshraghi, "Blood Cells Cancer (ALL) image dataset," Kaggle, 2021.
15. M. A. Rehman et al., "Computer-aided detection of ALL using deep learning," *Multimedia Tools Appl.*, vol. 82, pp. 42421–42440, 2023.
16. Y. Gal and Z. Ghahramani, "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning," in *Proc. ICML*, 2016, pp. 1050–1059.
17. C. Leibig et al., "Leveraging uncertainty estimates for predicting segmentation quality," arXiv:1612.01474, 2017.

ANNEXURE

A.1 Dataset

Blood Cells Cancer (ALL) dataset, Aria et al., Kaggle (`mohammadamireshraghi/blood-cell-cancer-all-4class`): 3,242 peripheral blood smear images, four classes.

A.2 Key Commands

Command	Purpose
python run.py	Launch the web UI at http://localhost:8000
python run.py --cli download	Download the dataset
python run.py --cli splits|train|ensemble|evaluate	Run a pipeline stage
python run.py --cli all --smoke	Run the whole pipeline on the sample (CPU)
python -m pytest -q	Run the automated tests

A.3 Main API Endpoints

Endpoint	Function
POST /api/predict	Predict a class with votes, flag and Grad-CAM overlay
POST /api/run/{stage}	Start a pipeline stage as a background job
GET /api/metrics	Read the evaluation metrics
GET /api/status	Report what exists on disk

Note: a research and educational project; not for clinical use.